2303a51618
vasu

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		
Program Name: B. Tech		**Assignment Typ**
Course Coordinator Name		Dr. Rishabh Mittal
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju
CourseCode	23CS002PC304	**Course Title**
Year/Sem	III/II	**Regulation**
Date and Day of Assignment	**Week7 – Wednusday**	**Time(s)**
Duration	2 Hours	**Applicable to Batches**
Assignment Number:13.3(Present assignment number)/**24**(Total number of assignments)		

	Question

1

Lab 13: Code Refactoring Using AI Assistance Improving Legacy Code for Readability, Maintainability, an

Lab Objectives

The objectives of this laboratory exercise are to:

- Identify common code smells and inefficiencies in legacy Python programs.
- Use AI-assisted coding tools to refactor code for better readability and maintainability.
- Apply modern Python best practices while preserving original program behavior.
- Analyze performance improvements achieved through refactoring.

Learning Outcomes

After completing this lab, students will be able to:

- Detect and eliminate code duplication in Python programs.
- Refactor inefficient logic using optimized data structures and constructs.
- Improve code quality through meaningful naming conventions and documentation.
- Validate refactored code using test cases and performance comparison.

Task 1: Refactoring – Removing Code Duplication

Objective

To eliminate repeated logic by extracting reusable functions.

Task Description

Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and

Starter (Legacy) Code

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Expected Outcome

- A reusable function to calculate area and perimeter
- No duplicated code blocks
- Proper docstrings for all functions

Removing Code Duplication Objective To eliminate repeated logic by extracting reusable functions. Task Description blocks of code calculating the area and perimeter of rectangles. Starter (Legacy) Code # Legacy script with repeated lo (5 + 10)) print("Area of Rectangle:", 7 * 12) print("Perimeter of Rectangle:", 2 * (7 + 12)) print("Area of Rectangle:", A reusable function to calculate area and perimeter No duplicated code blocks Proper docstrings for all functions

spark

Sure, I can help refactor that legacy Python script for you. I'll create a new function to calculate the area and perimeter c

Gemini

```python
def calculate_rectangle_properties(width, height):
    """
    Calculates and prints the area and perimeter of a rectangle.

    Args:
        width (int or float): The width of the rectangle.
        height (int or float): The height of the rectangle.

    Returns:
        tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = width * height
    perimeter = 2 * (width + height)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
    return area, perimeter

# Refactored calls using the new function
print("--- Rectangle 1 ---")
calculate_rectangle_properties(5, 10)

print("\n--- Rectangle 2 ---")
calculate_rectangle_properties(7, 12)

print("\n--- Rectangle 3 ---")
calculate_rectangle_properties(10, 15)
```

Sure, I can help refactor that legacy Python script for you. I'll create a new function to calculate the area and perimeter of a rectangle, reducing code duplication and adding appropriate docstrings.

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

```python
print("\n--- Rectangle 3 ---")
calculate_rectangle_properties(10, 15)
```

```
--- Rectangle 1 ---
Area of Rectangle: 50
Perimeter of Rectangle: 30

--- Rectangle 2 ---
Area of Rectangle: 84
Perimeter of Rectangle: 38

--- Rectangle 3 ---
Area of Rectangle: 150
Perimeter of Rectangle: 50
(150, 50)
```

Sure, I can help refactor that legacy Python script for you. I'll create a new function to calculate the area and perimeter of a rectangle, reducing code duplication and adding appropriate docstrings.

The refactored code has executed successfully! As you can see from the output, the `calculate_rectangle_properties` function correctly

Explain how the function returns values Capture the return values of the function for further

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")
```

Expected Outcome

- Optimized solution using set lookups or comprehensions
- Performance comparison before and after refactoring

AI ASST 13.3.ipynb

File Edit View Insert Runtime Tools Help

mmands + Code + Text Run all

Original (Legacy) Code and Performance Measurement

The original code uses nested loops to check if elements from `search_names` exist in `names`. This approach has a time complexity of O(n*m), where n is the length of `names` and m is the length of `search_names`. For large lists, this can become very inefficient.

```
import time

names = [f"Name_{i}" for i in range(10000)] # Larger list for better performance demonstration
search_names = [f"Name_{i}" for i in range(5000, 15000, 100)] # Some present, some not

print("--- Original (Legacy) Code ---")
start_time = time.time()

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
            break # Found, no need to continue inner loop
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")

end_time = time.time()
legacy_time = end_time - start_time
print(f"Original code execution time: {legacy_time:.6f} s
```

To improve performance by replacing inefficient nested loops with optimized structures. Task Description Use AI to analyze and refactor a script that checks the presence of elements using nested loops. Starter (Legacy) Code names = ["Alice", "Bob", "Charlie", "David"] search_names = ["Charlie", "Eve", "Bob"] for s in search_names: found = False for n in names:

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Refactored Code using Sets for Optimization

```
end_time = time.time()
refactored_time = end_time - start_time
print(f"Refactored code execution time: {refactored_time:.6f} seconds")
```

Gemini

Performance Comparison

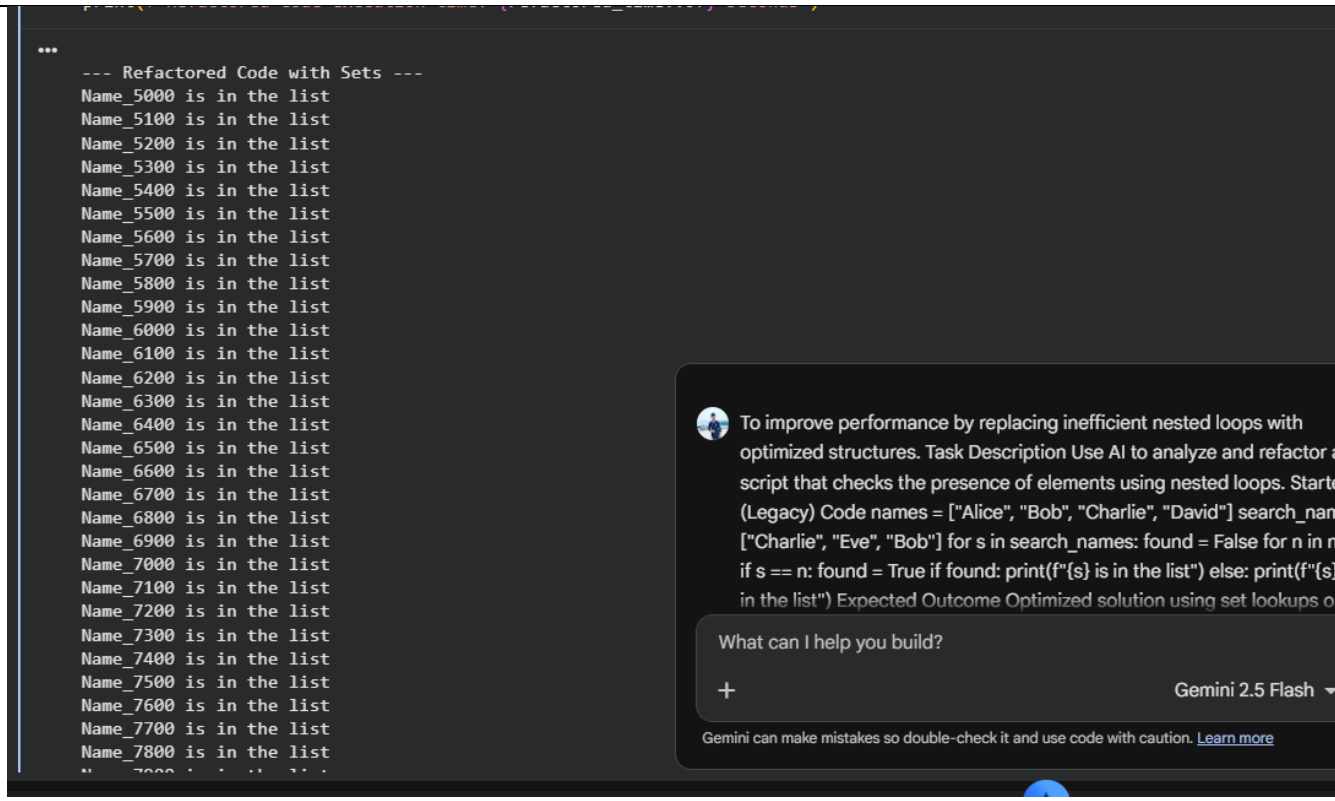
By comparing the execution times, you can observe the significant performance improvement achieved by using sets for element lookups. The set-based approach is considerably faster, especially as the size of the lists increases.

To improve performance by replacing inefficient nested loops with optimized structures. Task Description Use AI to analyze and refactor a script that checks the presence of elements using nested loops. Starter (Legacy) Code names = ["Alice", "Bob", "Charlie", "David"] search_names = ["Charlie", "Eve", "Bob"] for s in search_names: found = False for n in name

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash



Task 3: Refactoring – Extracting Reusable Functions

Objective

To modularize code by extracting calculations into reusable functions.

Task Description

Refactor a legacy script where price and tax calculations are written inline.

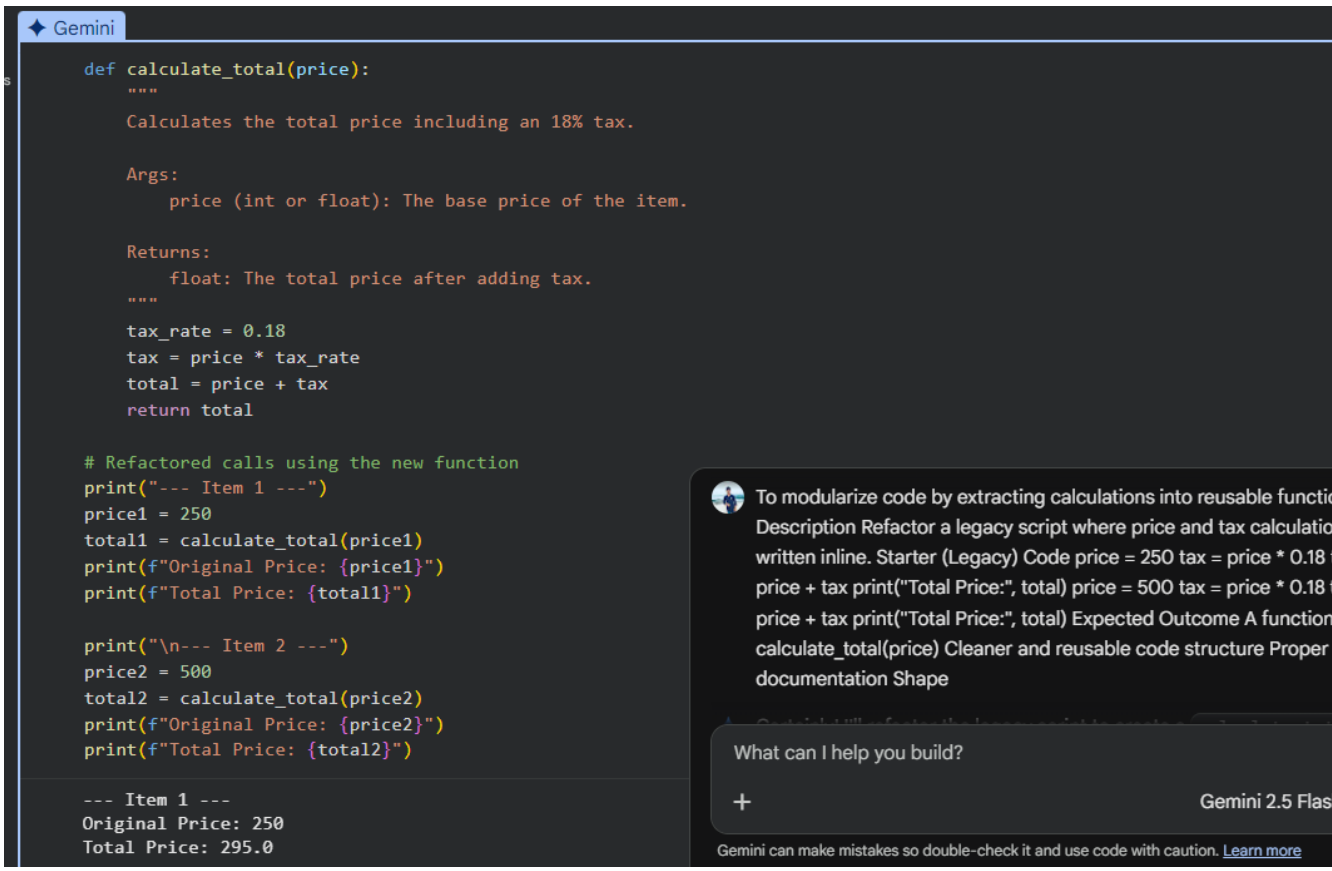
Starter (Legacy) Code

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)

Expected Outcome

- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation



```
--- Item 1 ---
Original Price: 250
Total Price: 295.0

--- Item 2 ---
Original Price: 500
Total Price: 590.0
```

Start coding or generate with AI.

Task 4: Refactoring – Replacing Hardcoded Values with Constants
Objective
To improve maintainability by replacing magic numbers with named constants.
Task Description
Use AI to identify hardcoded values and replace them with constants.
Starter (Legacy) Code

```
print("Area of Circle:", 3.14159 * (7 ** 2))
```

print("Circumference of Circle:", 2 * 3.14159 * 7)

Expected Outcome

- Constants such as PI and RADIUS
- Cleaner, maintainable code

Task 5: Refactoring – Improving Variable Naming and Readability

Objective

To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

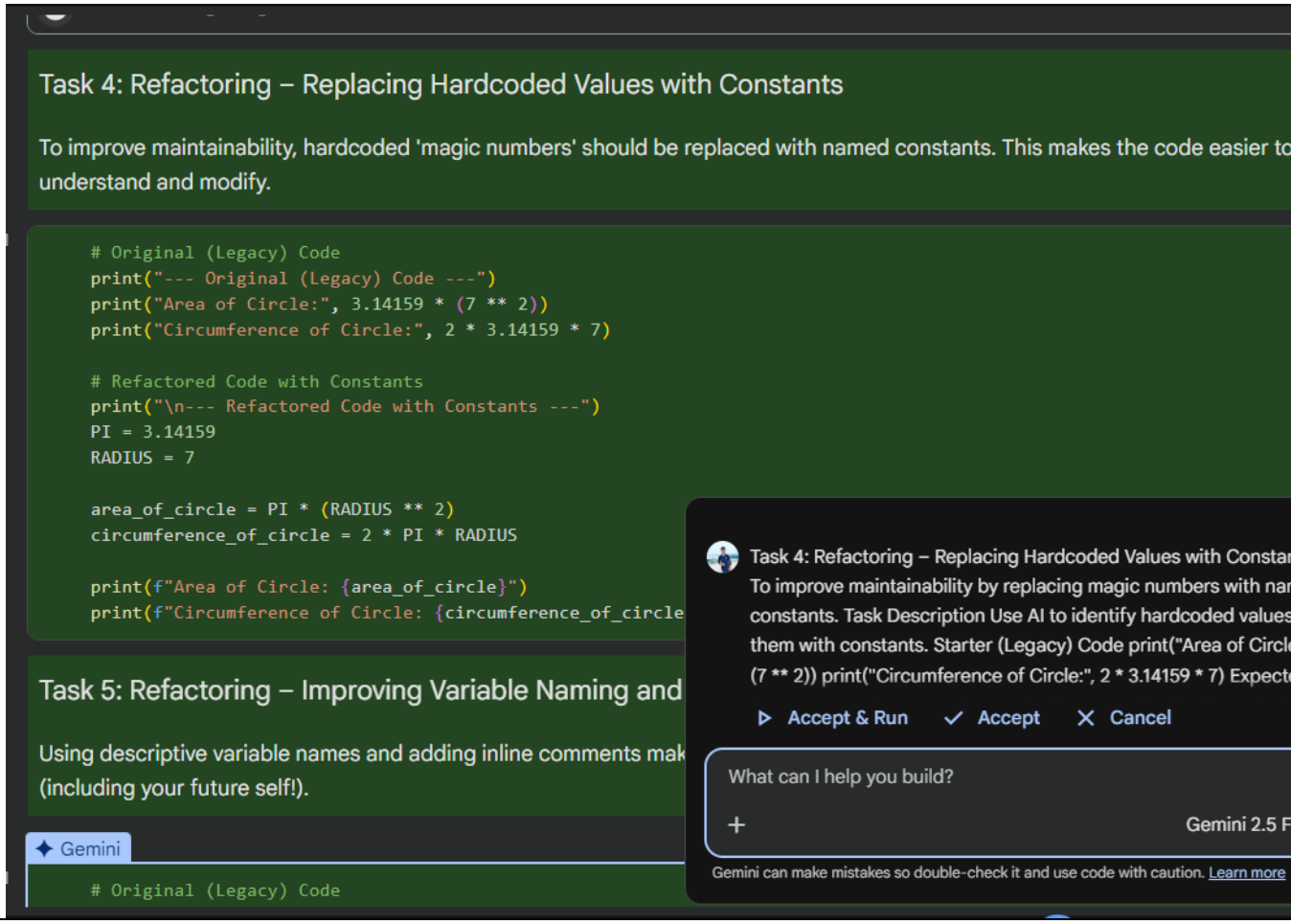
Starter (Legacy) Code

a = 10
b = 20
c = a * b / 2
print(c)

Expected Outcome

- Descriptive variable names
- Inline comments explaining logic
- Identical output

Note: Re
port should be submitted as a word documen



\

Task 5: Refactoring – Improving Variable Naming and Readability

Using descriptive variable names and adding inline comments makes code much easier to u
(including your future self!).

Gemini

```python
# Original (Legacy) Code
print("--- Original (Legacy) Code ---")
a = 10
b = 20
c = a * b / 2
print(f"Original output: {c}")

# Refactored Code with Improved Naming and Comments
print("\n--- Refactored Code with Improved Naming and Comments ---")
# Define the base and height of the triangle
triangle_base = 10
triangle_height = 20

# Calculate the area of the triangle
triangle_area = (triangle_base * triangle_height) / 2

print(f"Calculated triangle area: {triangle_area}")
```

```
--- Original (Legacy) Code ---
Original output: 100.0

--- Refactored Code with Improved Naming and Comments ---
Calculated triangle area: 100.0
```

Task 4: Refactoring –
To improve maintainab
constants. Task Descr
them with constants.
(7 ** 2)) print("Circum
Constants such as PI
Refactoring – Improvi

What can I help you buil

+

Gemini can make mistakes so dou

t for all tasks in a single document with prompts, comments & code explanation, and output and if required, screen